DRT Planning Toolkit Guidebook

1. Purpose and Analysis Workflow

Planning demand responsive transport (DRT) requires an assessment of both existing provision and the likely consequences of future service designs. Observed operations provide evidence about how resources are used and which outcomes are achieved, while prospective analysis examines how new services or alternative configurations might accommodate travel demand before implementation. The toolkit connects these perspectives through an integrated framework for *ex-post* performance evaluation and *ex-ante* operational simulation. It links evidence about existing services to the assessment of proposed services while distinguishing the roles of observation, prediction and simulation.

The analysis framework follows five stages: historical performance evaluation, analysis of performance associations, demand estimation, operational simulation and pooled comparative evaluation. Each stage addresses a specific planning question, as detailed in Table 1. Historical evaluation assesses the performance of existing DRT services, while performance association analysis identifies factors associated with performance differences. Demand estimation forecasts potential travel patterns, and operational simulation examines the feasibility of service configurations under specified resource and service constraints. Finally, the simulated services are evaluated alongside existing services using consistent performance indicators to assess their relative performance.

Table 1 Five modelling stages in the DRT toolkit

Stage	Inputs	Outputs/Application
Historical performance evaluation	Historical service resource inputs, outcomes, service types, intentions and prepared contextual indicators	Group and meta-frontier scores for benchmarking
Analysis of performance associations	Performance scores and service-level explanatory variables matched by `ServiceID`.	Estimates and diagnostics for interpreting performance associations
Demand estimation	Observed OD-hour demand, prepared explanatory features, scenario features and a validated fitted demand model	Explanatory relationships and diagnostics; hourly scenario forecasts for simulation
Operational simulation	Forecast demand, directed roads, grid access nodes, depots, fleet and service constraints	Served and rejected requests, waits, ride times, vehicle kilometres and vehicle activity
Pooled comparative evaluation	Historical and proposed indicators with compatible periods and definitions, costs and service intentions	Historical-only baselines and pooled comparative scores for service alternatives

Use this guidebook to prepare a new case study with service, demand and spatial data from the study area. Define the study objectives and the meanings of indicators before selecting models or running scripts. The framework can support different study areas, samples, and scenario sets, but transferring it to another case may require changes to formulas, identifiers, service classifications, calibration, and operating logic.

2. Workspace and Software

The toolkit provides two execution routes: a Windows PowerShell workflow and a desktop workflow using Python Jupyter notebooks and R scripts. Both routes use the same underlying analytical components. The choice concerns how the analysis is launched and managed, rather than a different modelling framework. Both routes use configuration files to define inputs, model settings, scenarios, and output locations, ensuring consistent and comparable analyses. Desktop execution supports interactive inspection of intermediate results, while PowerShell supports complete or repeated runs from a terminal.

The PowerShell workflow supports command-line execution and repeated or batch analyses. The `run_case.ps1` script runs the complete sequence using the selected Python interpreter, R executable and study configuration. Individual stages can also be executed through command-line instructions. When a separate software environment is required, the optional `setup.ps1` script prepares the Python environment and checks the required R packages. Progress is displayed in the terminal, and results and execution records are saved in the specified run folder.

The desktop workflow provides an interactive alternative through the notebooks and R scripts in the `desktop_scripts` directory. Users can execute notebook cells in Jupyter, source R scripts in RStudio, and inspect intermediate data, model estimates, and operational outcomes between stages. The combined notebook, `00_Run_All.ipynb`, runs the complete sequence, while the numbered entry files support individual stages.

2.1 Organise the workspace and software

Keep the folders `desktop_scripts`, drt_`toolkit`, `config`, and `data` together under a single toolkit root (`DRT_Toolkit/`). For each new study, create a dedicated subfolder within data and a separate configuration file, following the directory structure shown below.

```
DRT_Toolkit/
  config/project_A.json
  data/project_A/
    service_indicators.csv
    performance_explanatory.csv
    demand_observed.csv
    scenario_features/
    network.csv
    grid_nodes.csv
```

```
    depot_nodes.csv
    scenario_characteristics.csv
    cost_parameters.json
  desktop_scripts/
  drt_toolkit/
  outputs/project_A_001/
```

This layout illustrates file organisation, not a set of automatically generated inputs. Prepare the datasets required for each workflow and register their paths in the study configuration (see XX for instructions on configuring the study).

For the software version and required packages, this toolkit is tested to run on Python 3.10-3.12 interpreters, Jupyter, and R 4.4. The selected Python interpreter and R installation must include the dependencies required for the analysis. `requirements.txt` provides the reference dependency profile for Python, while R must be able to load the `AER`, `mgcv`, and `jsonlite` packages. If compatible installations are already available, users can simply select the appropriate Jupyter kernel and specify the path to the installed Rscript executable. Since the desktop scripts only check dependencies and do not install packages or create new environments, existing compatible environments can be used directly without requiring reinstallation.

2.2 Run through Windows PowerShell

The PowerShell route allows users to launch the analysis directly from a terminal without opening Jupyter or RStudio. To get started, open PowerShell in the toolkit root directory. If compatible Python and R installations are already available, the complete workflow can be executed using the following command:

```
.\run_case.ps1 `
  -Python 'C:\path\to\python.exe' `
  -Rscript 'C:\path\to\Rscript.exe' `
  -Config 'config/project_A.json' `
  -RunId 'project_A_001'
```

Replace the executable paths and study identifiers with the intended values. When using line continuation, ensure that the backtick is the final character on each continued line, with no trailing spaces. For this execution method, the configuration and run identifier must be provided as command-line arguments. Additionally, ensure that the R executable specified in the study configuration matches the R installation supplied to the wrapper. Alternatively, individual stages can be executed directly through the Python command-line interface, as illustrated in the following example:

```
& 'C:\path\to\python.exe' -m drt_toolkit evaluate `
  --config config/project_A.json `
  --run-id project_A_001
```

Use the same run identifier for all sequential stages within a single analysis to ensure consistency.

However, if the inputs, model settings, or analytical code are modified, a new run identifier must be assigned, as the command-line workflow prevents incompatible changes within an existing run folder.

2.3 Run through the desktop scripts

The desktop route uses the Jupyter notebooks and R scripts in `desktop_scripts`. Use Jupyter or a compatible editor for Python notebooks, and RStudio for interactive R code execution.

1. **Select the Python environment.** Open a notebook from within the toolkit folder and select a compatible Jupyter kernel. Kernel names depend on the local installation; they do not determine the analytical method.
2. **Select the study settings.** Edit `desktop_scripts/desktop_settings.json` to specify the study configuration, run identifier, and the installed location of `Rscript.exe`. For example:

```
{
  "config": "config/project_A.json",
  "run_id": "project_A_001",
  "rscript": "C:/path/to/Rscript.exe"
}
```

3. **Run the setup cell first.** It reloads the helper code, checks dependency compatibility and displays the selected run and output folder. After a toolkit update, reopen the notebook and run this cell.
4. **Execute the analysis.** Use `00_Run_All.ipynb` for the complete workflow or the numbered entry files for individual stages. Run notebook cells in order. The notebooks can call the R entries through the configured Rscript executable; the same R files can also be sourced in RStudio.

For RStudio execution after moving the toolkit, set `DRT_TOOLKIT_ROOT` before sourcing an entry file:

```
Sys.setenv(DRT_TOOLKIT_ROOT = "D:/DRT_Toolkit")
source(
  "D:/DRT_Toolkit/desktop_scripts/02_Performance_Drivers.R",
  encoding = "UTF-8"
)
```

Replace the installation path with the actual toolkit location. Alternatively, revise the `toolkit_root` setting in the R entry files. Python entries launched outside the toolkit folder may also require revision of their fallback root path.

The desktop scripts check dependencies but do not install packages or create environments. Saved settings are read before each stage call. When starting validation, evaluation or the complete sequence, the helper selects a fresh run identifier if the existing folder is incompatible with the current inputs or specifications, while preserving previous results.

The desktop scripts can be easily implemented individually within `desktop_scripts` folder. The preconditions and main outputs for each individual desktop stage are summarised in Table 2.

Table 2 Preconditions and main outputs for desktop stages

Entry stage	Preconditions	Main output used downstream
01_Performance_Evaluation .ipynb	Prepared historical table, supported service groups, indicators and service-intention mapping	`historical_scores.csv`
02_Performance_Drivers.R	Stage 1 scores and exactly matched explanatory records, with estimable formulas	Tobit coefficients, summaries, specifications and fitted objects
03_Demand_Estimation.R	Prepared observed demand and estimable configured formulas	Fitted demand relationships and diagnostic files
04_Demand_Prediction .ipynb	Compatible fitted demand model, scenario features and matching calibration/integrity settings	Hourly OD forecast tables and prediction summaries
05_Operational_Simulation .ipynb	Forecast demand, connected roads, matching grid/depot nodes and reviewed operating settings	`simulation_daily.csv`, `simulation_summary.csv` and passenger/vehicle detail
06_Service_Comparison .ipynb	Simulation or reviewed external monthly results, historical indicators, costs and comparability review	Proposed-service monthly indicators and pooled comparison results

For Stage 1, execute the input-inspection and performance-evaluation cells. The final optional cell can also launch the Stage 2 R script. Alternatively, to perform Stage 2 independently in RStudio, skip this optional cell and source `02_Performance_Drivers.R` once using the selected run ID. Stage 3 can be executed independently of the performance evaluation through its standalone R script. Notebook 04 also provides an optional cell to launch the same script. If demand estimation has already been completed in RStudio, skip this optional cell and proceed directly to the demand prediction cells. The resulting forecasts, associated with the selected run ID, are then used as inputs for Stage 5.

2.4 Creation of a dedicated environment [optional]

If a compatible existing environment is unavailable, use PowerShell to run the optional setup script from the toolkit root:

```
.\setup.ps1 `
  -Python 'C:\path\to\python.exe' `
  -Rscript 'C:\path\to\Rscript.exe'
```

The script creates `.venv`, installs its Python requirements, and checks or installs missing R packages in the selected R installation. Installation may require internet access. After setup, choose how to use the environment:

- **Desktop script**: register or select `.venv` as a Jupyter kernel.
- **Windows PowerShell**: pass `.\.venv\Scripts\python.exe` as the Python executable when

launching the workflow.

Environment setup is a separate activity from running the analysis and is unnecessary when compatible installations are already available.

3. Data Preparation

Before running the toolkit, assemble the data required for the stages of the study. These data fall into four connected groups: historical service records, observed and forecast demand, road-network and access information, and indicators for comparing proposed services with existing ones. Section 3.1 identifies the files required at each stage; Sections 3.2 to 3.5 explain how to prepare them; and Section 3.6 provides a final readiness check. The illustrative example is provided as reference material and is described separately in Appendix A. Its geographic scope, indicators, operating values, and assumptions are not general defaults and are not transferable to another study.

3.1 Dataset requirements by workflow stage

Table 3 presents the data requirements of each workflow stage in the toolkit. The workflow numbers in the table correspond to each modelling stage: 01 historical performance evaluation, 02 analysis of performance associations, 03 demand estimation, 04 operational simulation and 05 pooled service comparison. In the last column, ***Supply*** refers to study-specific inputs that users must prepare; ***Generated*** refers to files produced by an earlier stage and retained in the same run folder; and ***Conditional*** refers to files required only for the selected workflow alternative or when a non-null configuration path is specified.

Table 3 Dataset in each workflow stage

Workflow	Data	Contents and key fields	Source and preparation checks
01 & 05	Historical DRT operational and service table	Service ID, configured resource inputs and outcomes, service intention and contextual indicators.	***Supply***. One service and record per DMU; consistent measurement periods and units; finite, nonnegative indicator values.
01 & 05	Service intention mapping	Binary columns linking each intention to the selected policy operational and design outputs.	***Supply***. Each intention maps to at least one output. Names must match the flags and indicator columns in both historical and proposed-service tables.
02	Performance explanatory table	Service ID and all explanatory variables referenced by the selected Tobit formulas.	***Supply***. Exactly match historical service ID. Prepare categorical encodings and check missing or constant variables.
02	Performance scores	DMU, MetaEff, GroupEff and related score fields.	***Generated*** by 01.

03	Observed demand table	Grid ID (O/D), hour, observed trips, binary for zero trips, and all explanatory features required by the demand formulas.	***Supply***. Define the observation period, aggregation, feature units and transformations.
03	Scenario feature tables	One complete OD-hour feature table for each prediction case, using the reviewed model's feature definitions.	***Supply*** for the demand prediction. Cover the proposed origins, destinations and hours.
04	Forecast demand tables	Grid ID (O/D), hour and predicted trips, representing expected requests per OD pair during that hour.	***Generated*** by demand model in 03.
04	Directed road network	Nodes, edges, travel time (seconds) and distance (km).	***Supply***. Store consistent node IDs, and confirm directed connectivity.
04	Depot mapping	Valid depot node IDs and any associated grid or location attributes used by the depot table	***Supply***. Deployment uses the configured node list; all required depot and service nodes must be mutually routable.
04	Supporting spatial reference files	Node-coordinate tables, service-area or grid polygons, and case-specific spatial reference files where configured	***Conditional*** on configuration. Every non-null configured path must exist. Use consistent coordinate systems and inspect access-point snapping. Runtime routing uses the prepared road and grid-node tables.
04	Scenario definitions	Service ID, fleet size and shared operating settings in simulation scenarios and simulation.	***Supply*** as configuration. Give each alternative a unique ID distinct from historical services. Coordinate demand cases, fleet/capacity, hours, depots, constraints, seed and replications.
05	Proposed-service characteristics	Service ID, Service type, fleet size, service-intention flags and required contextual outputs	***Supply***. Match scenario IDs exactly. Prepare contextual indicators on the historical basis.
05	Cost parameters	Fuel cost, monthly driver wage and wage operating days for the implemented simulated-cost calculation	***Supply*** when using simulated cost accounting. Use a consistent currency and price basis. Align cost coverage with historical data; extend the

			accounting for additional components if needed.
05	Daily operational results	Simulation results with service ID and the trip-accounting fields used by the bridge	***Generated*** by 05. Retain scenario-replication records and check request conservation, chronology, waits and vehicle capacity before monthly conversion.
05	External monthly service indicators	Service ID, service type, required resource/outcome indicators, service-intention flags and contextual outputs for externally modelled proposals	***Conditional*** alternative to the built-in simulation and bridge. Complete the comparability review and run comparison separately. Use distinct IDs and the same reporting definitions as historical services.

Table 3 summarises the study-specific inputs and intermediate datasets passed between workflow stages. Other outputs, such as model summaries, fitted objects, passenger and vehicle details, and diagnostic figures, are generated for review and do not require manual preparation. The details of each data construction are illustrated in the following sections.

3.2 Common identifiers and service data

All input data should be stored as UTF-8 CSV files, with Service ID, Grid ID, and network Node ID treated as text fields. Use `ServiceID` consistently across historical, explanatory, and proposed-service datasets to ensure accurate data integration. Proposed service alternatives must be distinguishable from historical records, with each reconfigured service assigned a new identifier or version suffix. In the generated performance scores, the DMU field corresponds to the same service identifier and is mapped accordingly by the regression entry.

For historical data, prepare one record per consistently defined decision-making unit (DMU). Refer to the data dictionary for field definitions and select indicators relevant to the study objectives. Service-intention flags must be binary, with at least one active intention assigned to each service. Additionally, all indicator names specified in the intention mapping must be present in both the historical and proposed-service datasets.

To ensure data quality and comparability, use consistent measurement units and reporting periods across all datasets. Before running the analysis, check for duplicate records, missing values, invalid numeric ranges, and undefined or zero denominators. Any missing policy indicators should be addressed through documented data preparation or model-specification decisions rather than assigning arbitrary zeros solely to satisfy the required data interface.

3.3 Observed and prospective demand

Clearly define the demand observation period and document how individual travel records are

aggregated into origin-destination (OD) hourly observations. In particular, distinguish between fractional average demand and actual trip counts, as they represent different measures of travel demand. Prepare all service, demographic, destination, distance, and temporal features required by the estimation formulas and the selected prediction model. Ensure that variable names, measurement units, encodings, and transformations are consistent with the model specifications, as naming conventions alone do not guarantee that scaled and unscaled values are interchangeable.

For demand prediction, prepare scenario features for all required origin-destination-hour combinations, ensuring that each OD-hour key is unique unless the model explicitly accommodates repeated observations. The origin (`GridID_O`) and destination (`GridID_D`) identifiers must also be consistent with the access-node mapping. The predicted demand (`PredTrips`) represents the expected number of trip requests within a specified hour, rather than a per-second arrival rate. These hourly predictions are subsequently used by the simulator to generate individual trip requests.

All demographic and spatial-context features must be prepared at the appropriate service or OD level before execution. The toolkit does not automatically retrieve raw smartcard data, construct service catchment areas, or allocate population to service areas. Therefore, users must document the external data preparation procedures, including spatial units, reference dates, geographic coverage, and underlying assumptions. When applying an existing demand model to a new study area, its validity must be assessed against the target population and feature definitions. Simply retaining or reusing an example model does not establish its transferability.

3.4 Spatial and network data

Prepare the directed road network and depot definitions as a consistent set of spatial inputs. Verify that the road network accurately represents permitted travel directions, assumed speeds, and network connectivity. When deriving access points from polygons, use an appropriate projected coordinate reference system for distance calculations and examine snapping distances to ensure reasonable network connections. Note that the simulator uses the prepared access-node mapping directly and does not automatically generate access points or service catchment areas.

The reference input adapter converts numeric grid identifiers (`GridID`) from demand forecasts into an example-specific spatial prefix. If a different identifier system is used, the corresponding conversion logic in `drt_toolkit/simulation.py` must be reviewed and adjusted accordingly. Before running the simulation, ensure that all forecast grid identifiers and configured depot nodes correspond to valid road network nodes with feasible routes between them. Finally, note that the implementation calculates travel distances along the fastest-time routes, which may differ from routes that minimise distance.

3.5 Comparison data

Prepare the proposed-service characteristics using the same indicator definitions and contextual assumptions as the historical dataset to ensure comparability. The characteristics table must include

all fields required by the performance evaluation and comparison specifications, as well as those required by the data bridge. This bridge combines proposed-service characteristics with simulated operational totals to generate aggregate performance indicators (e.g., monthly data). Under the simulated trip-accounting approach, it derives trip and utilisation measures from dispatch tours and passenger-carrying tours and recalculates unit costs. However, the characteristics table must still contain all required fields, even when some indicators are subsequently derived from simulation results.

To ensure consistency in cost estimation, clearly document the currency, price year, and cost components used. The current implementation accounts for distance-related fuel costs and driver wages but does not provide a complete financial assessment. Depending on the study objectives, additional components, such as overheads, capital costs, fare revenue, and subsidies, may need to be incorporated or adjusted to maintain consistency with historical records. Other measures, including passenger-kilometres, delays relative to promised times, farebox revenue, and net subsidies, require additional data and calculations, as they are not included in the standard simulation outputs.

Before combining historical and simulated services for performance comparison, carefully review the consistency of reporting periods, indicator definitions, and measurement methods. Where definitions or measurement approaches differ, indicators should be harmonised before comparison. The same validation requirements apply to externally prepared aaggregate service datasets and those generated through the toolkit's built-in data bridge.

3.6 Check readiness before registering files

Confirm that all selected-stage inputs in the table are available, every configured path resolves, and no example file remains selected unintentionally. Check the historical and explanatory ID sets, scenario IDs and fleet sizes, forecast grid IDs, network-node coverage and depot settings together. Record units, reporting periods, feature transformations, preparation decisions and the source of each dataset. Keep generated inputs in the same run folder as their upstream stage. Register the prepared paths and assumptions using Section 4.

4. Case Study Configuration

The configuration file specifies input file locations, model specifications, and simulation scenario settings. To create a study-specific configuration, copy `config/project_template.json` to a new file, such as `config/project_A.json`. All configurations in the JSON file are summarised in Table 4. Note that the supplied template contains predefined reference settings rather than a blank configuration suitable for all studies. Therefore, carefully review and update all inherited file paths, model formulas, jurisdiction-specific settings, simulation scenarios, cost assumptions, and data integrity checks before execution. Preserve the existing configuration keys while replacing their values with those appropriate to the study and its prepared input data.

Table 4 Details of configuration area

Configuration area	Review before execution
paths	Historical, explanatory and demand tables; fitted demand model; scenario features; mapping, road, depot, spatial and cost inputs
evaluation & comparison_evaluation	Inputs, operational outputs and their relationship to the service-intention mapping
tobit.models	Group names, formulas and variables supported by the study sample
demand	Estimation formulas, threshold settings, prediction cases, model integrity and reference-forecast applicability
simulation	Scenarios, capacity, operating hours, depot nodes, constraints, priorities, seed and replications
bridge	Operating days, trip accounting, definition review and a study-specific comparability note

In the configuration file, review the demand-model path and scenario-feature entries as a coordinated interface. When demand cases change, update `prediction_cases`, their feature-file paths, each scenario `demand_case` and the spatial mappings together. Clear reference-demand paths to null unless study-specific reference forecasts are available. Do not compare a new study's predictions against the illustrative reference forecasts.

For desktop execution, edit `desktop_scripts/desktop_settings.json` before running a stage. After a toolkit update, reopen the updated notebook and run its first setup cell to refresh the helper code. The example below shows the structure; replace the R path with an installed executable and choose a new run ID:

```
{
  "config": "config/project_A.json",
  "run_id": "project_A_001",
  "rscript": "C:/path/to/Rscript.exe"
}
```

When using the current desktop helpers and standalone R scripts, retain `paths.outputs` as `outputs`, as these components expect results to be stored under `outputs/<run_id>` within the toolkit root directory. If a different output location is required, the shared Python helper, R path helper, and downstream notebook readers must be updated consistently. Changing the output path in the configuration file alone is insufficient to redirect all desktop components.

When adapting the toolkit to a study that differs from the reference implementation, review the following components and settings in Table 5 as needed. These represent potential areas for study-specific adjustments rather than additional models or procedures that are executed automatically.

Table 5 Study-specific adaptation requirements

Change required by the study	Files or components to review

Different service type classification	Group mapping in `drt_toolkit/evaluation.py` and related group formulas and service characteristics
Different explanatory variables or formula structure	Study configuration and both the standalone R entry and packaged R implementation for the affected model
Different demand features or calibration	Formulas and the fitted-demand component, validate and register a compatible fitted model
Different grid IDs, service logic or constraints	Input adapter and operational logic in `drt_toolkit/simulation.py`, plus the scenario settings
Different reporting period, costs or KPI definitions	Bridge comparison logic in `drt_toolkit/pipeline.py` and the prepared cost and service tables
Additional data inspection	Standard previews follow the selected configuration; use `get_study_config` for new inspection cells and review any custom fixed paths
Different study counts or validation targets	The example-only final audit cell and `tests/verify_case.py`; replace fixed reference counts with study-appropriate checks

5. Review Outputs and Validate the Application

Before interpreting the results, retrieve the output files from the selected run folder and verify that the run identifier, configuration, and input data versions are consistent with the intended analysis. Output patterns beginning with * represent model-generated files grouped by function, while the scripts preserve their predefined filename prefixes. The outputs and their associated review questions are summarised in Table 6.

Table 6 Modelling outputs and review questions

Output Group	Review Questions
Historical scores	Are service groups and indicator definitions correct? Which services form the relevant references?
Tobit sample, coefficients and specifications	Does the joined sample match? Are constants and formula changes documented? Is there adequate uncensored variation and stable estimation?
Demand estimates and diagnostics	Did estimation converge? Are feature effects plausible and basis checks satisfactory? Are in-sample fits distinguished from validation evidence?
`*_prediction_summary.csv` and `*_demand_CASE.csv.gz`	Are keys complete, rates finite and nonnegative, magnitudes plausible and scenario assumptions documented?
Simulation daily/summary and passenger/vehicle files	Do requests equal served plus rejected? Are capacity, chronology, waits, ride constraints, routing and vehicle distances consistent?
Monthly and pooled results	Are periods, costs, objectives and definitions comparable? Were historical-only and pooled frontiers calculated on the same primary specification?

Manifests, console files and session records	Were the intended inputs and software used? Did the required stages finish successfully? Which code and settings produced the results?

Performance evaluation should consider both operational outcomes and efficiency scores, as a service with high relative efficiency may still experience substantial unmet demand or excessive passenger waiting times. Efficiency scores should therefore be interpreted in the context of the selected sample and indicators, rather than as absolute measures of service quality. Similarly, Tobit regression coefficients should be interpreted as exploratory associations rather than evidence of causal relationships. Particular attention should be given to uncertainty in the estimated performance scores and to limitations associated with small sample sizes or limited variation in the data. Repeated simulations can help assess stochastic variability under the specified model assumptions and inputs, but they do not capture all sources of uncertainty related to demand, passenger behaviour, or future operating conditions.

For fixed-reference replication scores, retain and report all infeasibility statuses and their underlying reasons. Infeasibility may arise when a target lies beyond the fixed reference technology under the nonnegative-distance restriction. Failed estimates should not be replaced with zeros or excluded merely to produce apparently complete confidence intervals, as doing so could misrepresent the results. Finally, validate demand estimates against independent or held-out observations whenever possible. Road travel times, access-node locations, depot assumptions, and operational constraints should also be assessed for consistency with the study's actual conditions.

Appendix A

This appendix describes the data used at each stage, the meaning and units of primary fields, and the identifiers that connect the datasets. Taiwanese DRT cases are used as an example. The packaged example provides a reference for file structures and model execution. For a new application, prepare study-specific inputs and review the analytical assumptions before running the framework. Three types of information are used: historical service records for evaluation; OD-hour demand and spatial data for simulation; and proposed service characteristics for comparison, as detailed in Table 7.

Table 7 Analytical use of dataset

Dataset	One record represents	Analytical use	Path in the toolkit
Historical service indicators	One consistently defined existing service or DMU	Resource inputs, operational outcomes and contextual policy indicators for historical evaluation	`data/evaluation/ModelData.csv`
Service-intention mapping	One policy or service intention	Links intention flags to contextual evaluation outputs	`data/evaluation/si_mapping.csv`
Performance explanatory data	One historical service matched by `ServiceID`	Explanatory variables for performance associations	`data/tobit/tobit_socio.csv`
Observed demand	One origin-destination-hour observation on the declared reporting basis	Estimates relationships between travel demand, time and prepared contextual features	`data/demand/dt_model.csv`
Scenario feature table	One proposed origin-destination-hour combination	Supplies prospective conditions for demand prediction	`data/ml/dt_predict_200.csv.gz` `data/ml/dt_predict_500.csv.gz`
Road links and node references	One directed road link or one network node	Vehicle routing and spatial inspection	`data/simulation/network.csv.gz` `data/simulation/network_nodes.csv`
Depot tables	One depot location	Connects vehicle deployment to the road network	`data/simulation/grid_nodes.csv` `data/simulation/depot_nodes.csv`
Proposed-service characteristics	One proposed alternative	Supplies service attributes and contextual indicators for monthly evaluation	`data/scenarios/scenario_characteristics.csv`

Scenario settings and cost parameters	One configured alternative or a shared parameter set	Defines operating assumptions and monthly accounting	`config/case_study.json: simulation data/scenarios/cost_parameters.json`

Paths are relative to the toolkit root and describe the packaged example selected by `config/case_study.json`. Replace them with the registered paths for another study. In generated paths, `<run_id>` identifies the selected output folder and `<case>` identifies a demand case, such as 200 or 500. Scenario settings are stored in the configuration rather than in a separate dataset.

The packaged example contains 200 matched historical services from five jurisdictions (Tainan City, Kaohsiung City, Yilan County, Hualien County, Taitung County and New Taipei City) in Taiwan; 71,801 observed demand records; two prepared demand cases; and eight fleet alternatives. These quantities describe the example rather than requirements for another study.

The historical table is registered through `paths.historical`. The packaged file is `data/evaluation/Model Data.csv`. Each row represents one evaluation unit, with monthly operational indicators already prepared. The meaning of each field in the historical operational data is shown in Table 8.

Table 8 Fields of historical data (operational metric)

Field	Meaning	Unit or interpretation
`FleetNum`	Vehicles attributed to the service	Vehicles; resource input
`MonthlyTrips`	Monthly vehicle service trips, including trips carrying no passengers	Vehicle trips per month; resource input in the historical specification
`MonthlyPatronage`	Monthly passenger boardings	Boardings per month; desirable operational output
`MonthlyVKT`	Monthly vehicle kilometres travelled	Vehicle km per month; resource input
`KMCost`	Operating cost per vehicle kilometre	Local currency per vehicle km; unit cost, not monthly total cost
`Util_Rate`	Proportion of vehicle service trips carrying passengers	1 minus empty trips divided by total trips; fraction from 0 to 1
`TotalTrips_Fixed` & `TotalTrips_Booking`	Vehicle trips reported under fixed and booking operations	Already monthly averages in the supplied file
`TripsNoPax_Fixed` & `TripsNoPax_Booking`	Trips without passengers for each operation component	Vehicle trips per month
`VKT_Fixed` & `VKT_Booking`	Distance for each operation component	Vehicle km per month
`Patronage_Fixed` & `Patronage_Booking`	Boardings for each operation component	Boardings per month

The prepared contextual fields, presented in Table 9 describe the service area and its population or

destinations. Their geographic coverage and source date must be documented outside the model. The toolkit consumes these prepared values and does not run catchment analysis.

Table 9 Fields of historical data (service design metric)

Field	Meaning	Unit or interpretation
`Population`	Resident population associated with the service area	People
`DayworkPop`	Prepared daytime or work population	People; retain the source population definition
`KidsTeen, Elderly and Indigenous`	Population groups represented in the prepared service-area data	People; retain source age and population definitions
`School`	Schools associated with the service area	Count
`MedicalService`	Medical-service destinations associated with the service area	Count; its facility definition can differ from demand-table Hospital
`TransportHub, Amenity and Shop`	Transport hubs, amenities and shops	Counts under the declared source classifications
`TourismComment`	Prepared tourism review or comment measure	Contextual proxy; does not represent tourist boardings

Service-intention flags explain which policy dimensions apply to each service, as shown in Table 10. They are binary: 1 means the intention applies and 0 means it does not. At least one must apply to each service. The mapping is supplied in `data/evaluation/si_mapping.csv` through `paths.si_mapping` in configuration.

Table 10 Table of service intention mapping

Field	Meaning
`SI_LocalCentre`	Connection to local centres
`SI_Education`	Education-related service intention
`SI_Medicare`	Medical-access service intention
`SI_Elderly`	Service intention concerning older residents
`SI_Indigenous`	Service intention concerning Indigenous residents
`SI_Substitution`	Substitution-related transport intention
`SI_Tourism`	Tourism-related service intention

The explanatory dataset is registered through `paths.tobit_explanatory`; the supplied file is `data/tobit/tobit_socio.csv`. It contains the matched service records and prepared explanatory variables. Only variables referenced by the configured formulas are included in the regression. The current run supplies recalculated `MetaEff` and `GroupEff` from the historical evaluation.

The observed demand table, as shown in Table 11, is registered through `paths.demand_training`. The supplied `data/demand/dt_model.csv` represents mean boardings per OD-hour of a

representative day. Fractional Trips values are therefore expected. Scenario feature tables contain prospective conditions for the same type of OD-hour records, without requiring observed future Trips. `Suffix _O` denotes the origin and `_D` denotes the destination. Both sides must use consistent spatial definitions and feature units.

Table 11 Fields of demand estimation model

Field	Meaning	Unit or interpretation
`GridID_O, GridID_D & Hour`	Origin, destination and departure-hour key	IDs and integer hour 0 to 23; the example covers 6 to 22
`Trips`	Observed mean demand for an OD pair during that hour	Boardings per OD-hour on a representative day
`zero_trip`	Whether observed Trips equals zero	1 for zero Trips; 0 for positive Trips
`FixedRoute & DRTRoute`	Observation service-mode indicators	Binary; the prepared data use DRTRoute = 1 - FixedRoute
`TotalPop_O & TotalPop_D`	Prepared grid population	People
`Elderly_O & Elderly_D`	Prepared older population at each end	People; retain source age definition
`HighSchoolStudent_O & HighSchoolStudent_D`	Prepared high-school student population	People; retain source student definition
`TotalPop000_, Elderly000_ & HighSchoolStudent000_`	Scaled versions of the corresponding population fields, with _O or _D appended	Raw population divided by 1,000
`School_O & School_D`	Prepared school counts at each end	Counts; elementary, junior and senior fields retain component categories
`Hospital_O & Hospital_D`	Prepared hospital or medical destination category	Counts under the demand source's facility definition
`TransportHub_, Shop_ & Amenity_`	Prepared destination features, with _O or _D appended	Counts under source classifications
`Income_ave_ & Income_med_`	Prepared average and median income features at each end	Retain source currency, reference period and scale
`Income_med_10k_O & Income_med_10k_D`	Scaled median-income features	In the supplied data, Income_med divided by 10; verify original source units before reuse
`euclidist`	Straight-line distance between prepared grid centroids	Kilometres; differs from routed road distance
`FR_share`	Prepared conventional-route overlap indicator	Binary in the supplied data, despite the field name; not a continuous percentage

Downtown_O & Downtown_D	Whether each end lies in the declared central area	Source Boolean indicators

The fitted demand model and scenario features must be reviewed as a compatible set. The runtime checks the model file and recorded training-source identity. A newly observed demand file may therefore require model fitting and adaptation, rather than a path replacement alone.

Demand prediction generates an hourly OD table for each configured case. The primary fields in the demand prediction result are shown in Table 12. Independently prepared forecasts can use the same interface through paths.provided_od when the provided-demand route is selected.

Table 12 Fields of demand prediction results

Field	Meaning	Unit or interpretation
Hour	Departure hour represented by the forecast	Integer hour 0 to 23
GridID_O & GridID_D	Forecast origin and destination	Match the access-node mapping
PredTrips	Expected requests for this OD pair during the specified hour	Requests per OD-hour; finite and nonnegative, potentially fractional

The simulator draws individual requests from PredTrips and assigns request times within each hour. The supplied network and access files are data/simulation/network.csv.gz, grid_nodes.csv and depot_nodes.csv. The depot table uses GridID and node_id; actual vehicle deployment follows simulation.depot_nodes in the configuration.

The supplied data/scenarios/scenario_characteristics.csv, whose fields are listed in Table 13, is registered through paths.scenario_characteristics. It has one record for each proposed ServiceID and supplies characteristics not generated by the operational simulation. The fields of simulation scenarios

Table 13 Fields of scenario characteristics table

Field	Meaning	Relationship to other data
ServiceID & ServiceType	Proposed service identifier and evaluation group	Match configured alternatives; use a new ID for a reconfiguration
FleetNum	Vehicles assigned to the proposed service	Must equal fleet_size in its scenario settings
SI_ flags & contextual fields	Proposed service intentions, sociodemographic	Same definitions and required output fields as historical evaluation
demand_case	Forecast case used by an alternative	Stored in simulation.scenarios, rather than supplied as a required characteristics column
Buffer	Prepared example service-area descriptor	Metres in the supplied table; does not construct the service area

`fleet_size & capacity`	Number of vehicles and seats per vehicle	Scenario and shared simulation settings; distinct quantities

The example has cases 200 and 500, each with fleets of 2, 3, 4 and 5 vehicles. Labels and buffer metadata refer to prepared feature and spatial datasets.

Cost assumptions, listed in Table 14, are supplied through `paths.cost_parameters`, using `data/scenarios/cost_parameters.json` in the example. The implemented unit cost is `fuel_cost_per_km` plus the fleet size multiplied by the daily driver wage, divided by the mean daily VKT. It covers fuel and driver wages. Review other cost components and the common price basis before comparing with historical `KMCost`. Wage operating days and monthly service operating days are separate assumptions even when their example values coincide.

Table 14 Fields of cost assumptions table

Field	Meaning	Unit or interpretation
fuel_cost_per_km	Distance-related fuel cost	Currency per vehicle km; example 6.5
monthly_driver_wage	Monthly wage attributed to one vehicle's driver	Currency per month; example 40,000
wage_operating_days	Days used to convert monthly wages to a daily amount	Days; example 30
fare_per_passenger	Retained fare assumption	Currency per passenger; not used by the current bridge's KMCost calculation
operating_days_per_month	Multiplier converting representative daily operations to monthly indicators	Stored in bridge; example 30 days

In sum, prepare a study data dictionary before replacing inputs. For each selected field, record its definition, unit, observation period, geographic support, source, transformation and missing-value treatment. Preserve the same definitions across historical and proposed records. Do not substitute arbitrary zeros for missing contextual indicators. Review the configured formulas and indicator mapping against the available columns.

The packaged `docs/DATA_DICTIONARY.csv` provides a broader inventory of source fields. Some entries describe retained source or legacy columns absent from the current filtered tables. Use the active CSV headers and selected configuration to determine which fields are actually required. The definitions above prioritise fields used in the current framework and identify example-specific scaling and accounting choices.